

Assignment Module-38

Exception Handling

Q.1 In any programming language we categorise errors into 2 types

Syntax Error/CompileTime Mistake

Logical Error/RunTimeMistake

Syntax error/CompileTime Mistake

It refers to the mistakes done by the programmer with respect to syntax

These mistakes are identified by the compiler, so we say it as "CompileTimeMistake".

Logical Error/RunTimeMistake

It refers to the mistakes done by the programmer in terms of writing a logic

These mistakes are identified by JVM during the execution of a program, so we say it as "RunTimeMistake".

Q.2 An unwanted/expected event that disturbs the normal flow of execution of a program is called "Exception handling"

The main objective of Exception handling is to handle the exception

It is available for graceful termination of program.

Q.3 Exception handling can be performed using:

Try: the set of statements or code which requires monitoring for an exception is kept under this block.

Catch: this block catches all exceptions that were trapped in the try block.

Finally: this block is always performed irrespective of the catching of exceptions in the try or catch block.

Ex: class Launch

```
{
public static void main(String args[])
{
try
{s
System.out.print("Hello" + " " + 1 / 0);
}
catch(ArithmeticException e)
{
System.out.print("World");
}
}
}
```

Q.4. If there is no try and catch block while an exception occurs, the program will terminate. Exception handling ensures the smooth running of a program without program termination.

Q5. Errors typically happen while an application is running. For instance, Out of Memory Error occurs in case

the JVM runs out of memory. On the other hand, exceptions are mainly caused by the application. For instance,

Null Pointer Exception happens when an app tries to get through a null object.

Q.6 Based on handling by JVM, there are typically two types of exceptions in Java:

Checked: Occur during the compilation. Here, the compiler checks whether the exception is handled and throws an error accordingly.

Unchecked: Occur during program execution. These are not detectable during the compilation process.

Q.7 No, doing so will show a compilation error. Catch or finally block must always accompany try block. We can remove either finally block or catch block, but never both.

This will cause a **compilation error**, as a standalone **try** block is not allowed.

Ex:-

```
public class InvalidTryExample {
    public static void main(String[] args) {
        try {
            System.out.println("Executing try block");
        }
        // Compilation error: 'try' without 'catch', 'finally' or resource declarations
    }
}
```